TypeScript's best type inference based on the return value, which gives you some type protection:

```
function addThings(a: number, b: number) { return a + b }

let result: string = addThings(2, 3) // this will be a compiler error
```

In this case, TypeScript infers that the return value of the `addThings` function is a `number` because the returned value is the sum of two numbers. Later, assigning that value to the `result` variable, declared to be a string, will cause a compiler error even though the return value of `addThings` is not explicitly specified.

My recommendation is to be in the habit of explicitly specifying the return type in a function that returns values. Not only is specifying the return type better for communication, if you are relying on type inference, the type system won't catch if you accidentally return the wrong value or forget to return a value at all.

TypeScript function arguments behave differently than regular JavaScript in that the number of arguments to the function—the technical term is the *arity* of the function—is explicitly checked. So the following is valid JavaScript but invalid TypeScript:

```
function addThings(a, b) {
  return a + b
}

const result = addThings(2, 3, 4)
```

The function is declared with two arguments, but we call it with three arguments. TypeScript flags this as a compiler error even though we haven't specified any type information on any of this code. (In plain JavaScript, the third argument would be silently ignored.)

There are several legitimate cases where you might want to send different sets of arguments to the same function, and TypeScript offers a few different features to cover those cases.

First off, you can specify an argument as optional with the `?:` syntax, as in: